

LABORATORIO 09

Dónde quedaron tus flags

Un cliente Java por dentro, y por qué el broker no lo distingue de tu consola.

Guía para hacerlo por tu cuenta, paso a paso

Confluent Platform 8.2.0 · 3 brokers KRaft · Maven y Java 21

Duración estimada · 45 minutos

NovaTech Logistics · caso de estudio

Antes de empezar

Qué vas a lograr

Este laboratorio no es para que escribas código. Es para que el día que un desarrollador te mande un programa que «no conecta con Kafka», puedas abrir el archivo, encontrar las cuatro líneas que importan, y decirle cuál está mal.

Eso es trabajo de administración. Y no se puede hacer sin haber visto nunca un cliente por dentro.

Al final vas a haber producido mensajes con un programa Java, y los vas a haber leído con **exactamente el mismo comando de consola del laboratorio 06**, sin adaptar nada.

LO QUE ESTE LABORATORIO DEMUESTRA

Lo que hiciste con dos comandos de consola son veinte líneas de código, y hace exactamente lo mismo.

Si al final dices «es mi comando de siempre, escrito de otra forma», el laboratorio cumplió.

Qué necesitas tener listo

Requisito	Cómo lo compruebas
Docker Desktop corriendo, con 6 GB de RAM	Ajustes de Docker Desktop, pestaña <i>Resources</i>
Git Bash abierto en la carpeta del lab	<code>cd Capitulo_4/lab-09-clientes-java-spring</code>
Maven instalado	<code>mvn -v</code> tiene que contestar con un número de versión. Es la única pieza de este laboratorio que no vive en Docker
Los puertos 9092, 9093, 9094 y 8090 libres	El paso 1 te avisa si no lo están
Haber hecho el laboratorio 06	Hoy se compara el código Java contra los comandos de aquel día

COMPRUEBA MAVEN ANTES DE EMPEZAR, NO EN EL PASO 4

Escribe esto ahora:

```
mvn -v
```

Si contesta `bash: mvn: command not found`, los pasos 4 en adelante no corren. **Los pasos 1 al 3 sí**, y son la mitad del laboratorio. Lee el rescate del paso 4 antes de decidir qué haces.

El caso

Un martes te llega un correo del equipo de desarrollo de **NovaTech Logistics**. Dice que el servicio de comprobantes no está publicando en Kafka, y pregunta si pueden revisar el clúster.

Revisas el clúster. Los tres brokers están arriba. El tópico existe, con sus particiones y sus réplicas al día. No hay nada que revisar, porque **el clúster está perfecto**.

Y aquí la conversación se puede ir por dos caminos.

En el primero contestas «por aquí está todo bien» y devuelves el correo. Dos días después el problema sigue, ahora con más gente copiada, y nadie ha mirado el único lugar donde puede estar.

En el segundo pides el archivo. Lo abres. Son treinta líneas y veinticinco no te importan. Las que importan son cuatro, y las reconoces porque son las mismas cosas que tú escribiste con guiones en el laboratorio 06. Encuentras que el `bootstrap.servers` apunta a un puerto interno que desde la máquina del desarrollador no existe, y contestas eso.

La diferencia entre los dos caminos no es saber Java. Es haber visto una vez que el código de un cliente Kafka no tiene nada nuevo adentro.

Y LA SEGUNDA MITAD, QUE ES LA QUE DUELE

El broker no puede ayudarte a distinguir. Para él, un cliente Java y una consola son el mismo cliente. No hay un log que diga «este mensaje vino de una aplicación». Si el mensaje no llegó, el broker no tiene nada que contarte.

LA METÁFORA · HOY ENTRA LA CAJA REGISTRADORA

Kafka sigue siendo **el restaurante**. El **mozo** es el broker, el **tipo de comanda** es el tópico, los **sectores del salón** son las particiones, el **cocinero** es el consumidor y la **brigada** es su grupo.

Hoy **no se suma ninguna pieza nueva**, y ésta es justamente la lección.

Hasta ahora, cuando querías meter una comanda, la escribías tú, a mano, en el talonario. Eso era `kafka-console-producer`. Un cliente Java es **la misma comanda escrita por una caja registradora** en vez de por una mano.

La caja tiene que saber lo mismo que sabía la mano. A qué mozo entregarle la comanda, de qué tipo es, y si lleva número de mesa. Ni más ni menos.

Y el mozo no sabe si la comanda la escribió una mano o una máquina. Le llega, la clava en el pincho, y sigue. No tiene forma de saberlo, y por eso no te lo puede decir.

LA REGLA QUE SALE DE AHÍ

Un cliente no es una capa sobre Kafka. **Es un cliente de Kafka, igual que la consola.** Lo que cambia es quién teclea.

Levantar el clúster

Qué vamos a hacer

Encender los tres brokers y crear el tópico de trabajo. Es el mismo arranque de los laboratorios anteriores y no trae nada nuevo.

El comando

```
cd Capitulo_4/lab-09-clientes-java-spring
chmod +x bin/*.sh
bin/start-lab.sh
```

chmod +x bin/*.sh — le da permiso de ejecución a los scripts. En Windows los archivos llegan sin ese permiso, así que esta línea hace falta **una sola vez**

bin/start-lab.sh — levanta los tres brokers, espera a que los tres queden *healthy*, levanta Kafbat UI y crea el tópico `novatech.lab09.pedidos` con 3 particiones y factor de réplica 3

EL FINAL DE LO QUE VAS A VER · RECORTADO

```
[2/4] Esperando a que los 3 brokers estén operativos...
✓ 3/3 brokers operativos

[3/4] Esperando a que Kafbat UI esté disponible...
✓ Kafbat UI disponible

[4/4] Inicializando tópico de trabajo...
[OK] Clúster disponible
Created topic novatech.lab09.pedidos.
✓ Tópico 'novatech.lab09.pedidos' creado (3 particiones, RF 3)
```

CLÚSTER NOVATECH LAB 09 OPERATIVO

Componentes:

Broker 1:	localhost:9092
Broker 2:	localhost:9093
Broker 3:	localhost:9094
Kafbat UI:	http://localhost:8090

UN AVISO QUE SALE ANTES DEL «CREATED TOPIC»

Aparece una línea larga que empieza con `WARNING: Due to limitations in metric names...` y habla de puntos y guiones bajos. **Sale siempre y no es un problema.** Kafka avisa de que un nombre de tópico con puntos podría chocar con otro en los nombres de sus métricas. El del curso no choca con nada.

VAS BIEN SI...

Hay cuatro contenedores arriba y los tres brokers dicen `healthy`. Compruébalo:

```
docker ps --filter "label=com.docker.compose.project=novatech-lab09" \
  --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'
```

NAMES	STATUS	PORTS
kafbat-ui	Up 2 minutes	0.0.0.0:8090->8080/tcp, [::]:8090->8080/tcp
kafka-broker-3	Up 2 minutes (healthy)	0.0.0.0:9094->9094/tcp, [::]:9094->9094/tcp
kafka-broker-1	Up 2 minutes (healthy)	0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp
kafka-broker-2	Up 2 minutes (healthy)	0.0.0.0:9093->9093/tcp, [::]:9093->9093/tcp

Y el tópico tiene 3 particiones, factor 3, y las tres réplicas de cada partición en ISR:

```
docker exec kafka-broker-1 kafka-topics \
  --bootstrap-server kafka-broker-1:29092 \
  --describe --topic novatech.lab09.pedidos
```

```
Topic: novatech.lab09.pedidos   TopicId: _Lm85W0eRrafKN2uVvW61A PartitionCount: 3
ReplicationFactor: 3           Configs: min.insync.replicas=2
      Topic: novatech.lab09.pedidos Partition: 0   Leader: 2           Replicas: 2,3,1
Isr: 2,3,1                     Elr:      LastKnownElr:
      Topic: novatech.lab09.pedidos Partition: 1   Leader: 3           Replicas: 3,1,2
Isr: 3,1,2                     Elr:      LastKnownElr:
      Topic: novatech.lab09.pedidos Partition: 2   Leader: 1           Replicas: 1,2,3
Isr: 1,2,3                     Elr:      LastKnownElr:
```

El `TopicId` tuyo va a ser otro. Se genera al crear el tópico. Lo que tiene que calzar es el `PartitionCount: 3`, el `ReplicationFactor: 3` y las tres réplicas en cada `Isr`.

SI DICE QUE EL PUERTO YA ESTÁ EN USO

El síntoma es una línea de Docker que nombra `Bind for 0.0.0.0:9092 failed: port is already allocated`. Casi siempre es otro laboratorio del curso que quedó arriba.

Mira quién lo tiene y bájalo:

```
docker ps --filter "label=com.docker.compose.project" \
  --format '{{.Names}}\t{{.Label "com.docker.compose.project"}}'
```

SI DICE DOCKER NO ESTÁ CORRIENDO

Abre Docker Desktop y espera a que el icono deje de moverse. El script comprueba esto antes que nada y sale sin tocar nada.

SI SE QUEDA EN 2/3 BROKERS LISTOS Y SE ACABA EL TIEMPO

Es memoria. Tres brokers en 4 GB arrancan justos y a veces uno no llega.

Sube Docker Desktop a 6 GB en *Settings* → *Resources*, y vuelve a correr `bin/start-lab.sh`.

Los flags que ya sabes

Qué vamos a hacer

Antes de abrir un archivo Java hay que tener fresco con qué se lo va a comparar. Éste es el comando con el que produjiste en el laboratorio 06, tal cual.

NO LO EJECUTES · ESTE COMANDO SE MIRA

El tópic `novatech.validacion` es del laboratorio 06 y **aquí no existe**. En el lab 06 lo creaste tú a mano, y este laboratorio no lo crea. **Está escrito para que lo leas.**

Si lo ejecutas por error **no rompes nada**, pero lo que pasa merece que lo veas, porque es una trampa que se paga cara en producción. Está en el rescate de abajo.

SI LO EJECUTASTE, Y POR QUÉ VALE LA PENA MIRAR LO QUE SALIÓ

No se queda colgado. **Tarda 61 segundos, termina solo, y esto es lo último que imprime:**

```
[2026-08-29 04:57:09,374] ERROR Error when sending message to topic novatech.validacion
with key: 15 bytes, value: 13 bytes with error:
(org.apache.kafka.clients.producer.internals.ErrorLoggingCallback)
org.apache.kafka.common.errors.TimeoutException: Topic novatech.validacion not present in
metadata after 60000 ms.
Caused by: org.apache.kafka.common.errors.UnknownTopicOrPartitionException: This server
does not host this topic-partition.
```

Antes de eso repite unas sesenta veces un `WARN` con

`{novatech.validacion=UNKNOWN_TOPIC_OR_PARTITION}`, uno por segundo. **El mensaje nunca se entregó.**

Y ahora lo que de verdad importa, medido. El código de salida de ese comando es **cero**, que en la línea de comandos significa éxito.

```
rc del docker exec = 0      ·      duracion = 61 segundos
```

Un guion que solo mire el código de salida daría este envío por bueno. Perdió el mensaje y reportó éxito. Es exactamente el laboratorio de hoy visto desde el otro lado, o sea que el clúster estaba perfecto y el problema estaba en el cliente. **Cuando automatices envíos, no te fíes del código de salida. Comprueba el offset.**

El comando del laboratorio 06

```
echo "RUC-20100066601:comprobante_1" | \
docker exec -i kafka-broker-1 kafka-console-producer \
  --bootstrap-server kafka-broker-1:29092 \
  --topic novatech.validacion \
  --property parse.key=true --property key.separator=:
```

`docker exec -i` — ejecuta un comando dentro de un contenedor que ya corre. La `-i` mantiene la entrada abierta, que es lo que deja que el `echo` de la izquierda le llegue al productor. **Aquí no va `-t`**, porque no hay una persona tecleando

`--bootstrap-server` — a qué clúster hablarle. Cualquier broker sirve, porque el primero al que le preguntas te presenta a los demás

`--topic` — a qué tópico escribir

`--property parse.key=true` — que la línea que escribes trae clave y valor pegados

`--property key.separator=:` — con qué carácter se parten

Quédate con las cuatro decisiones que tomaste al escribirlo, porque son las que vas a buscar en el archivo Java.

Lo que decidiste	Con qué flag
A qué clúster hablarle	<code>--bootstrap-server</code>
A qué tópico escribir	<code>--topic</code>
Si el mensaje lleva clave	<code>--property parse.key=true</code>
Cómo separar la clave del valor	<code>--property key.separator=:</code>

VAS BIEN SI...

Puedes decir en voz alta, sin mirar la tabla, qué decidiste con cada uno de los cuatro flags. **Son cuatro decisiones, no cuatro palabras que se copian.** El paso 3 no funciona si no las tienes.

El mismo comando, escrito en Java

Qué vamos a hacer

Abrir el archivo del productor y buscar en él tus cuatro flags. **No hay que entender Java para esto.** Hay que reconocer nombres.

CONCEPTO · PROPERTIES

Una lista de pares **clave igual valor** que se le entrega al cliente antes de arrancarlo. **Son tus flags.** Cada `--algo` que escribiste en el laboratorio 06 es una línea de esta lista, con otro nombre.

CONCEPTO · KAFKAPRODUCER

El objeto que abre la conexión y habla el protocolo de Kafka. Es lo que en la consola era el comando entero, o sea `kafka-console-producer`.

CONCEPTO · PRODUCERRECORD

Un mensaje. A qué tópico va, con qué clave, y con qué valor. En la consola esto era **una línea de texto** que tú escribías y que partía el separador.

CONCEPTO · SERIALIZADOR

Kafka **solo mueve bytes**. No sabe qué es un pedido, ni un número, ni una fecha. El **serializador** es la función que convierte tu objeto en bytes al escribir, y el **deserializador** la que los vuelve a convertir al leer. En la consola nunca lo viste porque el texto ya era bytes.

El comando

```
cat cliente-java/src/main/java/com/novatech/kafka/ProductorApp.java
```

cat — imprime un archivo completo en la terminal. **Aquí no hay ningún docker adelante**, porque el archivo está en tu disco, no dentro de un contenedor

Salen **53 líneas**. Puedes contarlas:

```
wc -l cliente-java/src/main/java/com/novatech/kafka/ProductorApp.java
```

LO QUE VAS A VER

```
53 cliente-java/src/main/java/com/novatech/kafka/ProductorApp.java
```

De esas 53, estas son las que importan. Son las únicas que vamos a leer.

```
Properties props = new Properties();
props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, BOOTSTRAP);
props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());
props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
PedidoSerializer.class.getName());
props.put(ProducerConfig.ACKS_CONFIG, "all");

try (Producer<String, Pedido> producer = new KafkaProducer<>(props)) {
    ProducerRecord<String, Pedido> record =
        new ProducerRecord<>(TOPIC, pedido.pedidoId(), pedido);
    producer.send(record, (metadata, exception) -> { ... });
    producer.flush();
}
```

La tabla que es el laboratorio entero

Cada flag que escribiste en el paso 2 está aquí, con otro nombre.

Lo que escribiste en el lab 06	Dónde está en el Java
<code>--bootstrap-server kafka-broker-1:29092</code>	<code>props.put(BOOTSTRAP_SERVERS_CONFIG, BOOTSTRAP)</code>
<code>--topic novatech.validacion</code>	El primer argumento de <code>new ProducerRecord<>(TOPIC, ...)</code>
<code>--property parse.key=true</code>	El segundo argumento , o sea <code>new ProducerRecord<>(TOPIC, pedido.pedidoId(), pedido)</code>
<code>--property key.separator=:</code>	No existe, y no puede existir. En la consola clave y valor viajaban pegados en una línea de texto y había que partirla. En Java son dos parámetros distintos, así que no hay nada que separar
<i>no tenía flag</i>	<code>props.put(ACKS_CONFIG, "all")</code> , que es el <code>acks</code> del laboratorio 07, aquí escrito a mano
<i>no tenía flag</i>	Los dos <code>SERIALIZER_CLASS_CONFIG</code> . En la consola nunca aparecieron porque lo que escribías ya era texto. Aquí hay que decir cómo se convierte un <code>Pedido</code> en bytes

Ese es el archivo entero. Lo demás son el bucle, el `System.out.printf` y los `import` . Si mañana te llega un programa que no conecta, **estas cinco líneas son las que abres**, y la primera de la lista es la que falla nueve de cada diez veces.

LA PREGUNTA QUE VALE · CONTÉSTALA ANTES DE SEGUIR

La línea 19 del archivo dice esto:

```
private static final String BOOTSTRAP =
"localhost:9092,localhost:9093,localhost:9094";
```

¿Por qué no dice `kafka-broker-1:29092` , que es lo que usa la consola?

LA RESPUESTA, Y ES LA QUE VAS A USAR EN SUNAT

Porque **la consola corre dentro de la red de Docker y este programa corre fuera**, en tu máquina.

Un nombre de contenedor como `kafka-broker-1` no se resuelve fuera de esa red. Y una dirección `localhost` no sirve dentro de ella, porque ahí `localhost` es el propio contenedor. **Es exactamente la clase de error que vas a diagnosticar.**

VAS BIEN SI...

Encontraste tus cuatro flags en el archivo sin copiar de la tabla, y puedes explicar por qué el cuarto no está. **Anota una respuesta.** ¿Cuántas de las 53 líneas son configuración de Kafka?

```
SI CAT DICE NO SUCH FILE OR DIRECTORY
```

Estás en la carpeta equivocada. Comprueba con `pwd` que la terminal termina en `lab-09-clientes-java-spring`. La ruta del comando es relativa a esa carpeta, no a `cliente-java/`.

Correrlo

Qué vamos a hacer

Compilar el programa y ejecutarlo. No hay nada que escribir, porque el proyecto ya está hecho.

El comando

```
cd cliente-java
mvn -q compile exec:java \
  -Dexec.mainClass="com.novatech.kafka.ProducorApp" \
  -Dexec.args="5"
```

mvn — Maven, la herramienta con la que se construyen los proyectos Java. **Es la única dependencia de este laboratorio que no está en Docker**

-q — modo silencioso. Sin esto Maven imprime treinta líneas de su propio proceso antes de llegar a lo tuyo

compile — convierte el `.java` en `.class`. **La primera vez descarga las bibliotecas**, que son 16 MB en 25 archivos

exec:java — ejecuta una clase del proyecto

-Dexec.mainClass=... — cuál de las clases. El proyecto tiene un productor y un consumidor, y aquí se pide el productor

-Dexec.args="5" — los argumentos del programa. Aquí, cuántos pedidos enviar

LO QUE VAS A VER

```
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
Enviado a novatech.lab09.pedidos [particion 1, offset 0]
Enviado a novatech.lab09.pedidos [particion 1, offset 1]
Enviado a novatech.lab09.pedidos [particion 2, offset 0]
Enviado a novatech.lab09.pedidos [particion 2, offset 1]
Enviado a novatech.lab09.pedidos [particion 0, offset 0]
Total de pedidos enviados: 5
```

LAS TRES PRIMERAS LÍNEAS SON RUIDO Y SALEN SIEMPRE

SLF4J es la biblioteca de registro de Java diciendo que no encontró con qué escribir logs, y que se conforma con no escribir ninguno. **No es un error del laboratorio** y no afecta a nada. Está escrito aquí para que no te distraiga.

Cómo se lee

Lo que dice	Qué significa
<code>Enviado a</code> <code>novatech.lab09.pedidos</code>	El tópico al que fue. El mismo tipo de nombre que vienes usando desde el laboratorio 05
<code>[particion 1, offset 0]</code>	Kafka le contestó al programa. No es el programa adivinando. Es el broker confirmando dónde quedó el mensaje. Eso pasa porque el <code>acks=all</code> de la quinta línea lo obligó a esperar la confirmación
Los cinco repartidos entre las tres particiones	El reparto por clave . Cada pedido lleva un UUID como clave, y su hash decide la partición

A TI TE VAN A SALIR OTROS NÚMEROS DE PARTICIÓN

La clave es un UUID que el programa genera en cada corrida, así que el reparto cambia. En tres corridas medidas salieron `2,2,1,1,1`, `2,2,2,2,0` y `1,1,2,2,0`. **Lo que no cambia nunca es que el broker le dijo al programa dónde quedó cada uno.** Eso es lo que hay que mirar.

VAS BIEN SI...

Salieron cinco líneas `Enviado a...` y la última dice `Total de pedidos enviados: 5`. Cada línea trae su partición y su offset.

Anota una respuesta. ¿Quién le dijo al programa en qué partición quedó cada mensaje, el programa o el broker? ¿Y qué línea del archivo obliga a esperar esa respuesta?

SI DICE `BASH: MVN: COMMAND NOT FOUND`

Maven no está instalado, y es la única cosa de este laboratorio que no vive en Docker.

Este paso y el 4 no corren, pero el laboratorio no se acaba aquí. Los pasos 2, 3 y 7 se hacen igual, y son los que de verdad te van a servir en SUNAT, porque son leer un archivo y diagnosticar un grupo.

Si quieres instalarlo, Maven necesita además un Java 17 o superior. En una VM Windows sin permisos de administrador puede que no puedas, y no pasa nada. **Salta al paso 7** y usa los mensajes que otro compañero haya producido, o produce cinco a mano con el `kafka-console-producer` del laboratorio 06.

SI SE QUEDA COLGADO Y TERMINA CON `TIMEOUTEXCEPTION`

El programa no encontró el clúster. Las dos causas, en orden de frecuencia:

Una. El clúster no está arriba. Vuelve al paso 1 y comprueba los cuatro contenedores.

Dos. Los puertos 9092, 9093 y 9094 no están publicados hacia tu máquina. El programa corre fuera de Docker y entra por ahí. Compruébalo en la columna PORTS del `docker ps` del paso 1.

SI MAVEN FALLA DESCARGANDO DEPENDENCIAS

Es la red. La primera compilación baja 16 MB del repositorio central de Maven, y detrás de un proxy corporativo puede no salir. Vuelve a intentarlo, porque Maven guarda lo que ya bajó y el segundo intento arranca donde se quedó el primero.

Leerlo con la consola del laboratorio 06

Qué vamos a hacer

Éste es el paso que cierra la afirmación. Si un cliente Java fuera «otra cosa», haría falta un consumidor Java para leerlo.

Vamos a usar **el mismo comando de consola del laboratorio 06**, sin adaptar nada.

CONCEPTO · OFFSET

El número de orden de un mensaje dentro de su partición. **Empieza en 0 y solo sube**. Nunca se reutiliza, ni siquiera cuando el mensaje se borra por retención.

El comando

```
cd ..
docker exec kafka-broker-1 kafka-console-consumer \
  --bootstrap-server kafka-broker-1:29092 \
  --topic novatech.lab09.pedidos \
  --from-beginning --max-messages 5 \
  --formatter-property print.key=true
```

cd .. — vuelve a la carpeta del laboratorio. El paso anterior te dejó dentro de `cliente-java/`

docker exec — sin `-i` ni `-t` esta vez, porque no le vas a escribir nada al comando. **Solo lee y escupe**

--from-beginning — empieza por el mensaje más viejo que el tópico conserva

--max-messages 5 — **sin esto el comando no termina nunca**. Lee cinco y sale solo

--formatter-property print.key=true — imprime la clave además del valor. Es el reemplazo moderno de `--property`, que en Kafka 8 avisa de que está en desuso

LO QUE VAS A VER

```

97486ebe-9a8d-48b5-b617-0d0822767e2e    {"pedidoId":"97486ebe-9a8d-48b5-b617-
0d0822767e2e","cliente":"cliente-3","producto":"producto-3","cantidad":3,"monto":300.0}
8254b5be-f506-4e06-8cd4-f7102969426e    {"pedidoId":"8254b5be-f506-4e06-8cd4-
f7102969426e","cliente":"cliente-1","producto":"producto-1","cantidad":1,"monto":100.0}
bb334d9a-e3a5-481b-8cd5-3c9a5f37225d    {"pedidoId":"bb334d9a-e3a5-481b-8cd5-
3c9a5f37225d","cliente":"cliente-5","producto":"producto-0","cantidad":5,"monto":500.0}
f418e007-db7f-4eba-a368-ad3b9c8b9695    {"pedidoId":"f418e007-db7f-4eba-a368-
ad3b9c8b9695","cliente":"cliente-2","producto":"producto-2","cantidad":2,"monto":200.0}
28e00754-6b87-4367-b08d-1438cf028733    {"pedidoId":"28e00754-6b87-4367-b08d-
1438cf028733","cliente":"cliente-4","producto":"producto-4","cantidad":4,"monto":400.0}
Processed a total of 5 messages

```

Los UUID son distintos en cada corrida, porque los genera el programa. Los `cliente-N` y los montos no cambian.

Cómo se lee

Lo que se ve	Qué demuestra
A la izquierda del tabulador, el UUID	Es la clave que puso el <code>new ProducerRecord<>(TOPIC, pedido.pedidoId(), pedido)</code> . La consola la lee sin saber quién la escribió
A la derecha, el JSON	Es lo que produjo el <code>PedidoSerializer</code> . Para Kafka son bytes. Que se vean como JSON legible es una decisión del programa , no de Kafka
El comando es el del lab 06	No existe un <code>kafka-console-consumer --java</code> . No hace falta, y ahí está la afirmación

VAS BIEN SI...

Salieron cinco líneas con un UUID, un tabulador y un JSON, y la última dice `Processed a total of 5 messages`.

Y no tuviste que cambiar una sola letra del comando del laboratorio 06. Eso es lo que había que ver.

SI EL COMANDO SE QUEDA COLGADO Y NO VUELVE

Se te olvidó el `--max-messages 5`. El consumidor de consola se queda esperando mensajes nuevos para siempre. Sal con `Ctrl+C` y vuelve a escribirlo completo.

SI SALE UN AVISO SOBRE `--PROPERTY` EN DESUSO

Escribiste `--property print.key=true` en vez de `--formatter-property print.key=true`.
Funciona igual, pero Kafka 8 avisa de que la forma vieja va a desaparecer. Usa la nueva.

Por qué la segunda vez no trae nada

Qué vamos a hacer

Aquí llega el matiz que muerde, y que es la causa número uno de «el consumidor dejó de leer» en producción.

Alguien va a decir que la consola tiene `--from-beginning` y el Java tiene `auto.offset.reset=earliest`, y que no son lo mismo. **Sí lo son.** Lo que engaña es otra cosa.

CONCEPTO · GRUPO DE CONSUMIDORES

Varios consumidores que comparten un nombre de grupo y se reparten las particiones de un tópico. **Kafka guarda por dónde va el grupo**, no cada consumidor. Ese avance guardado se llama *offset comprometido*, y sobrevive a que el consumidor se apague.

Los cuatro comandos

Córrelos en este orden y anota cuántos mensajes trae cada uno.

```
# A · sin grupo, dos veces
docker exec kafka-broker-1 kafka-console-consumer \
  --bootstrap-server kafka-broker-1:29092 \
  --topic novatech.lab09.pedidos \
  --from-beginning --timeout-ms 10000
```

```
# B · con grupo, dos veces, el MISMO grupo
docker exec kafka-broker-1 kafka-console-consumer \
  --bootstrap-server kafka-broker-1:29092 \
  --topic novatech.lab09.pedidos \
  --group mi-grupo-de-prueba \
  --from-beginning --timeout-ms 10000
```

`--timeout-ms 10000` — espera diez segundos sin recibir nada y se va. **Aquí reemplaza al `--max-messages`**, porque hay corridas en las que no va a llegar ningún mensaje y hay que poder verlo

`--group mi-grupo-de-prueba` — le pone nombre al grupo. **Ésta es la única diferencia entre A y B**, y es la que cambia el resultado

LO QUE SALE · LAS CUATRO CORRIDAS MEDIDAS

```
A · sin grupo, 1ª vez -> Processed a total of 5 messages
A · sin grupo, 2ª vez -> Processed a total of 5 messages
B · con grupo, 1ª vez -> Processed a total of 5 messages
B · con grupo, 2ª vez -> Processed a total of 0 messages
```

LAS CUATRO CORRIDAS TERMINAN CON UN `ERROR`, Y NINGUNA FALLÓ

Con `--timeout-ms`, el comando siempre termina así:

```
[2026-08-29 03:19:58,671] ERROR Error processing message, terminating consumer process:
(org.apache.kafka.tools.consumer.ConsoleConsumer)
org.apache.kafka.common.errors.TimeoutException
Processed a total of 0 messages
```

Eso no es un fallo. Es el `--timeout-ms` cumpliéndose. El consumidor esperó diez segundos, no llegó nada más, y se fue. **La línea que importa es la última**, la del `Processed a total of`. Medido, la corrida que devuelve 0 tarda 11 segundos.

Cómo se lee

El consumidor de consola, cuando no le das `--group`, **se inventa un grupo nuevo en cada arranque**. Como ese grupo nunca leyó nada, no tiene offsets guardados, y `earliest` lo manda al principio. Por eso parece que `--from-beginning` siempre lee todo.

En cuanto le pones un grupo de verdad, que es lo que hace cualquier programa, **se comporta igual que el Java. La segunda vez no trae nada.**

POR QUÉ ESTO IMPORTA EN SUNAT

Es la causa número uno de «el consumidor dejó de leer» en producción. **El grupo ya tenía offsets guardados y nadie los miró.** El tópico se llena, el consumidor está vivo, y no trae nada porque ya leyó hasta ahí.

VAS BIEN SI...

Tu tabla dice **5, 5, 5, 0**. Y puedes explicar por qué las dos primeras dieron lo mismo y las dos últimas no. **Anota una respuesta.** Las cuatro llevan `--from-beginning`. ¿Por qué solo una devolvió cero?

SI LA SEGUNDA CORRIDA DE B TE TRAE 5 OTRA VEZ

Cambiaste el nombre del grupo entre una y otra, o se te olvidó el `--group`. **Tiene que ser el mismo nombre exacto en las dos.** Cópialo, no lo escribas de nuevo.

SI QUIERES VOLVER A LEERLO TODO DESDE EL PRINCIPIO

Usa un nombre de grupo que no hayas usado nunca. Un grupo nuevo no tiene offsets guardados, así que `--from-beginning` vuelve a funcionar. **Ésa es la prueba de que el problema eran los offsets y no el tópico.**

Diagnosticarlo, que es lo que harás en SUNAT

Qué vamos a hacer

El paso anterior fabricó el problema. Éste lo diagnostica, con un comando que ya conoces del laboratorio 06.

CONCEPTO · LAG

Cuántos mensajes hay escritos que el grupo todavía no leyó. Es la resta entre el offset final del tópico y el offset por donde va el grupo. **Es el número que se vigila en producción**, más que ningún otro.

El comando

```
docker exec kafka-broker-1 kafka-consumer-groups \
  --bootstrap-server kafka-broker-1:29092 \
  --describe --group mi-grupo-de-prueba
```

kafka-consumer-groups — el comando que administra grupos. Sin él, un grupo es una caja negra

--describe — muestra dónde va el grupo, partición por partición

--group — cuál grupo. **Tiene que ser el mismo nombre exacto del paso 6**

LO QUE VAS A VER

Consumer group 'mi-grupo-de-prueba' has no active members.

GROUP	TOPIC	PARTITION	CURRENT-OFFSET	LOG-END-OFFSET	LAG
CONSUMER-ID	HOST	CLIENT-ID			
mi-grupo-de-prueba	novatech.lab09.pedidos	0	1	1	0
-	-	-			
mi-grupo-de-prueba	novatech.lab09.pedidos	1	2	2	0
-	-	-			
mi-grupo-de-prueba	novatech.lab09.pedidos	2	2	2	0
-	-	-			

Cómo se lee

Lo que dice	Qué significa
<code>has no active members</code>	Nadie está consumiendo ahora mismo. El grupo existe igual , porque dejó su marca guardada
<code>CURRENT-OFFSET</code>	Por dónde va el grupo en esa partición. Esto es lo que sobrevivió entre la primera y la segunda corrida del paso 6
<code>LOG-END-OFFSET</code>	Hasta dónde llegó el tópico en esa partición
<code>LAG 0</code> en las tres	El grupo ya leyó todo. Ahí está, en números, la razón de que la segunda corrida devolviera cero

Los repartos de tu corrida van a ser otros, porque las claves son UUID distintos. Lo que tiene que calzar es que **los tres** `CURRENT-OFFSET` **suman 5** y que los tres `LAG` están en cero.

LO LOGRASTE SI...

La suma de los tres `CURRENT-OFFSET` te da 5, los tres `LAG` están en cero, y puedes contar la historia completa. El programa Java produjo cinco. La consola sin grupo los leyó dos veces. El grupo los leyó una vez y guardó por dónde iba. La segunda vez no había nada nuevo.

Anota una respuesta. Un consumidor de producción dejó de recibir mensajes y el tópico sigue llenándose. Antes de tocar el clúster, ¿qué miras y con qué comando?

SI DICE QUE EL GRUPO NO EXISTE

El síntoma es `Consumer group 'x' does not exist`. Escribiste otro nombre, o nunca corriste el bloque B del paso 6. **Un grupo se crea al usarlo, no antes.** Lista los que hay con `--list` en vez de `--describe --group`.

Lo que aprendiste

1 Un cliente Kafka no es una capa sobre Kafka. Es Kafka.

El broker no distingue una consola de una aplicación, y no hay log que te lo diga. Lo comprobaste leyendo con el comando del laboratorio 06 lo que escribió un programa Java, sin adaptar una letra.

2 Cuando un programa «no conecta», el primer archivo que se abre es el de las propiedades.

Y la primera línea que se mira es `bootstrap.servers`. Nueve de cada diez veces está ahí. Además esa dirección depende de dónde corre el programa, porque un nombre de contenedor no se resuelve fuera de la red de Docker y un `localhost` no sirve dentro de ella.

3 El serializador y el deserializador son dos piezas que nadie obliga a coincidir.

Viven en dos programas distintos. Si el que escribe cambia y el que lee no se entera, el que se cae es el que lee, dos días después. Eso es lo que el laboratorio 11 viene a cerrar con Schema Registry.

4 Un consumidor con grupo no vuelve a leer lo que ya leyó.

Por más `earliest` o `--from-beginning` que tenga. Si un consumidor «dejó de recibir», mira sus offsets antes que el clúster, con `kafka-consumer-groups --describe --group`.

Para profundizar

Todo esto estaba en el recorrido de clase y salió por tiempo. Aquí va desarrollado, con su comando.

A · El consumidor Java, y su grupo

El paso 5 leyó con la consola a propósito. El proyecto también trae su consumidor.

```
cd cliente-java
mvn -q compile exec:java \
  -Dexec.mainClass="com.novatech.kafka.ConsumidorApp" \
  -Dexec.args="grupo-java-nativo"
```

-Dexec.args="grupo-java-nativo" — el nombre del grupo. Aquí el argumento es el grupo, no la cantidad de mensajes como en el productor

LO QUE SALE · LAS PRIMERAS LÍNEAS

```
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
Consumidor en grupo 'grupo-java-nativo' escuchando novatech.lab09.pedidos (Ctrl+C para salir)...
[particion 2 offset 0] cliente-2 -> producto-2 x2 ($200.00)
[particion 2 offset 1] cliente-4 -> producto-4 x4 ($400.00)
[particion 1 offset 0] cliente-1 -> producto-1 x1 ($100.00)
[particion 1 offset 1] cliente-5 -> producto-0 x5 ($500.00)
[particion 0 offset 0] cliente-3 -> producto-3 x3 ($300.00)
```

No termina solo. Sal con `Ctrl+C`.

Lo que hay que mirar. Sus propiedades son cinco y son las mismas de siempre, más

`GROUP_ID_CONFIG`, que es el `--group` del laboratorio 06, y `AUTO_OFFSET_RESET`. Córrelo dos veces con el mismo grupo y verás el 5 y el 0 del paso 6, con un programa en vez de una consola.

B · La serialización, de verdad

```
cat cliente-java/src/main/java/com/novatech/kafka/PedidoSerializer.java
cat cliente-java/src/main/java/com/novatech/kafka/PedidoDeserializer.java
cat cliente-java/src/main/java/com/novatech/kafka/Pedido.java
```

Lo que hay que mirar. El serializador es, en esencia, objeto a bytes. El deserializador es bytes a objeto. **Son dos archivos separados que nadie obliga a coincidir.** Si el productor agrega un campo y el consumidor no se entera, el que revienta es el consumidor.

La pregunta que vale. ¿Qué pasaría si el productor mandara `monto` como texto y el consumidor lo esperara como número? Guarda la respuesta, porque el laboratorio 11 hace exactamente esa pregunta.

C · Spring for Apache Kafka

La misma cosa, con el código escondido detrás de anotaciones.

```
cd cliente-spring
mvn spring-boot:run
```

Y en otra terminal, produciendo por HTTP:

```
curl -X POST -H "Content-Type: application/json" \
  -d '{"pedidoId":"p-1001","cliente":"ACME","producto":"caja","cantidad":3,"monto":1500.0}' \
  http://localhost:8081/api/pedidos
```

Lo que hay que mirar. `KafkaTemplate` hace lo que hacía el `KafkaProducer` a mano, y `@KafkaListener` reemplaza el bucle de `poll()`. Las propiedades no desaparecieron. Se mudaron a `src/main/resources/application.yml` y a la clase `KafkaConfig`.

Y AHÍ ESTÁ EL PUNTO PARA UN ADMINISTRADOR

Cuando el programa es Spring, **las propiedades están en un `.yaml`, no en el `.java`**. Ése es el archivo que hay que pedir.

D · Grupos y rebalanceo, con procesos de verdad

Abre tres terminales en `cliente-java/`, las tres con el mismo grupo:

```
mvn -q compile exec:java \
  -Dexec.mainClass="com.novatech.kafka.ConsumidorApp" \
  -Dexec.args="grupo-escalado"
```

Y en una cuarta, produciendo:

```
mvn -q compile exec:java \
  -Dexec.mainClass="com.novatech.kafka.ProductorApp" \
  -Dexec.args="30"
```

Lo que hay que mirar. El tópico tiene 3 particiones, así que las tres instancias reciben una cada una. Mata una con `Ctrl+C` y vuelve a producir. Sus particiones se reparten entre las dos que quedan. **Es el rebalanceo del laboratorio 06, con procesos en vez de terminales.**

La pregunta que vale. ¿Qué pasa si levantas una cuarta instancia sobre 3 particiones?

E · El desafío de interoperabilidad

Produce con `cliente-java` y consume con la aplicación de Spring corriendo. Documenta si el consumidor de Spring recibió lo que produjo la API nativa, y qué dice eso sobre el formato del mensaje en el tópico.

F · El reporte del laboratorio

`plantillas/reporte-entregable.md` recorre las actividades con sus preguntas. Las respuestas de referencia están en `soluciones/reporte-resuelto.md`.

Antes de cerrar

DEJA EL TERRENO LIMPIO

```
bin/90-test-lab.sh    # ver el estado del laboratorio
bin/stop-lab.sh      # apagar los contenedores
```

`bin/stop-lab.sh` solo detiene los contenedores. No borra nada, y los volúmenes siguen ahí mientras el lab esté detenido.

Ojo al reanudar. `bin/start-lab.sh` reconstruye el laboratorio desde cero, así que los tópicos y mensajes que creaste no sobreviven. El lab vuelve a sembrar los suyos. **Si creaste algo propio, anótalo antes.** Para borrarlo todo ahora mismo, `bin/reset-lab.sh`, que no pide confirmación.

LO QUE DICE EL VALIDADOR SI TODO ESTÁ EN ORDEN

```
Validador del Lab 09 – estado actual
✓ los 3 brokers están healthy
✓ el tópico novatech.lab09.pedidos existe
✓ cliente-java ya compilado (target/ presente)
✓ cliente-spring ya compilado (target/ presente)

✓ LAB EN BUEN ESTADO (4 verificaciones OK)
```

LO QUE TE LLEVAS

De los incidentes de Kafka que tu equipo atendió el último año, ¿cuántos terminaron siendo del clúster y cuántos de la configuración de un cliente?

Hoy viste por qué la segunda cifra suele ser la grande, y por qué el clúster no te va a ayudar a encontrarlos. **El broker no distingue quién le escribe.** Esa respuesta está en el archivo del cliente, y ahora sabes qué cinco líneas mirar.